# Data Structures and Algorithms

INFORMATION
**TECHNOLOGY**

## Objectives

- In this session, you will learn to:
  - Sort data
  - Sort data by using bubble sort
  - Sort data by using insertion sort

## Sorting Data

- Consider that you need to retrieve the telephone number of a person named Steve from a telephone directory where the names are stored randomly.
- To retrieve the desired record, you need to sequentially traverse the list of names one by one because the names are not sorted. This is a time-consuming activity.
- A simple solution to save time, and search records efficiently is sorting.
- Sorting is the process of arranging data in some predefined order or sequence.
- If the data is ordered, you can reduce the number of records to be traversed.

## Selecting a Sorting Algorithm

- To select an appropriate algorithm, you need to consider the following factors:
  - Execution time
  - Storage space
  - Programming effort
- When the data is large, you need to select a sorting algorithm that makes most efficient use of time or memory.

## Types of Sorting Algorithms

- There are various sorting algorithms that are used to sort data. Some of these are:
  - Bubble sort
  - Insertion sort
  - Quick sort

## Sorting Data by Using Bubble Sort

- Bubble Sort Algorithm:
  - Is one of the simplest sorting algorithms.
  - Has a quadratic order of growth and is therefore suitable for sorting small lists only.
  - Works by repeatedly scanning through the list, comparing adjacent elements, and swapping them if they are in the wrong order.

**Let us view an animation to understand the implementation of the bubble sort algorithm.**

## Implementing the Bubble Sort Algorithm

- Algorithm for bubble sort:

```
1. Set pass = 1.
2. Repeat step 3 varying j from 0 to n - 1- pass.
3. If the element at index j is greater than the
   element at index j + 1, swap the two elements.
4. Increment pass by 1.
5. If pass <= n - 1, go to step 2.
```

## Determining the Efficiency of the Bubble Sort Algorithm

- The efficiency of a sorting algorithm is measured in terms of number of comparisons.
- There are `n-1` comparisons in Pass 1, `n-2` comparisons in Pass 2, and so on.
- The total number of comparisons will be calculated by using the following formula:

```
Sum = (n - 1) + (n - 2) + (n - 3) + … + 3 + 2 + 1
Sum = n/2 [2a + (n - 1)d]
Sum = (n - 1)/2 [2 × 1 + (n - 1 - 1) × 1]
Sum = (n  - 1)/2 [2 + (n - 2)]
Sum = (n - 1)/2 (n)
Sum = n(n - 1)/2
```

- The bubble sort algorithm is of the order $O(n^2)$.

## Just a Minute

- What is the order of growth for the bubble sort algorithm?
  - Linear
  - Loglinear
  - Constant
  - Quadratic

# Data Structures and Algorithms

## Just a Minute (Contd.)

- Solution:
  - Quadratic

## Just a Minute (Contd.)

- While implementing the bubble sort algorithm, how many comparisons will be performed in Pass 1?
  - `n-1` comparisons
  - `n-2` comparisons
  - `n-3` comparisons
  - `n-4` comparisons

## Just a Minute (Contd.)

- Solution:
  - `n-1` comparisons

## Demo: Sorting Data by Using the Bubble Sort Algorithm

- Problem Statement:
  - Write a program to store the marks of 10 students in an array. Include a function to sort the elements of the array by using the bubble sort algorithm. After sorting the array, display the sorted list.

## Demo: Sorting Data by Using the Bubble Sort Algorithm (Contd.)

- Solution:
  - To sort the marks of students by using the bubble sort algorithm, you need to perform the following tasks:
    1. Write the code.
    2. Execute the code and verify the output.

## Sorting Data by Using Insertion Sort

- Insertion Sort Algorithm:
  - Has a quadratic order of growth and is therefore suitable for sorting small lists only.
  - Is much more efficient than bubble sort, if the list that needs to be sorted is nearly sorted.

**Let us view an animation to understand the implementation of the insertion sort algorithm.**

## Implementing the Insertion Sort Algorithm

- Algorithm for insertion sort:

```
1. Repeat steps 2, 3, 4, and 5 varying i from 1 to
   n - 1.
2. Set temp = arr[i].
3. Set j = i - 1.
4. Repeat until j becomes less than 0 or arr[j] becomes
   less than or equal to temp:
     a. Shift the value at index j to index j + 1.
     b. Decrement j by 1.
5. Store temp at index j + 1.
```

## Determining the Efficiency of the Insertion Sort Algorithm

- To sort a list of size `n` by using insertion sort, you need to perform `(n - 1)` passes.
- In `n - 1` passes, you need to make `n - 1` comparisons. This is the best case for insertion sort.
- Therefore, the best case efficiency of insertion sort is of the order, `O(n)`.
- If the list is stored in the reverse order, you need to make `n - 1` comparisons in the $(n - 1)^{th}$ pass.
- The formula for determining the total number of comparisons in this case is:

```
Sum = 1 + 2 + . . . + (n - 1)
```

- Therefore, the worst case efficiency of insertion sort is same as that of bubble sort, that is, $O(n^2)$.

## Just a Minute

- Which one of the following options signifies the worst case efficiency of the insertion sort algorithm?
  - $O(n^2)$
  - $O(2^n)$
  - $O(n)$
  - $O(n \log n)$

## Just a Minute (Contd.)

- Solution:
  - $O(n^2)$

## Demo: Sorting Data by Using the Insertion Sort Algorithm

- Problem Statement:
  - Write a program that stores 10 numbers in an array, and sorts them by using the insertion sort algorithm.

## Demo: Sorting Data by Using the Insertion Sort Algorithm (Contd.)

- Solution:
  - To sort a list of numbers by using the insertion sort algorithm, you need to perform the following tasks:
    1. Write the code.
    2. Execute the code and verify the output.

## Summary

- In this session, you learned that:
  - Sorting is the process of arranging data in some predefined order or sequence.
  - There are various sorting algorithms that are used to sort data. Some of these are:
    - Bubble sort
    - Insertion sort
    - Quick sort
  - Bubble sort algorithm has a quadratic order of growth and is therefore suitable for sorting small lists only.
  - In bubble sort, there are `n - 1` comparisons in Pass 1, `n - 2` comparisons in Pass 2, and so on.

## Summary (Contd.)

- The insertion sort algorithm divides the list into two parts, sorted and unsorted.
- Insertion sort performs a different number of comparisons depending on the initial ordering of elements.
- When the elements are already in the sorted order, insertion sort needs to make few comparisons.